



Amrita School of AI
COIMBATORE CAMPUS

24AIM332 · INTRODUCTION TO CLOUD COMPUTING

Course Handbook

How this course runs, what you will build, and the two live industry systems it is anchored to.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

COURSE

24AIM332 Introduction to Cloud Computing

STRUCTURE

L-T-P-C 3-0-2-4

CONTACT

3 lecture hours + one 2-hour lab, weekly

DURATION

15 teaching weeks

OFFERING

Odd Semester 2026-27

What this course is about

Cloud computing is usually taught as a tour of service names. That produces students who can recite the difference between IaaS and PaaS and cannot deploy anything.

This course is built the other way round. You will learn the concepts, and you will also build and destroy real infrastructure from code, package an application into a container, orchestrate it, and wire a pipeline that takes a commit all the way to a running system with no human step in the middle.

Two extension topics are deliberately added beyond the printed syllabus, **infrastructure as code** and **CI/CD**, because without them “deploying to the cloud” means clicking in a console, and nothing you build can be rebuilt.

The course is anchored to two systems that are actually running, presented as case studies with the engineering mistakes left in.

Learning outcomes

OUTCOME	
CO1	Explain cloud service and deployment models and select an appropriate one for a workload
CO2	Distinguish hypervisor and container virtualization and apply each correctly
CO3	Develop and deploy a web application using public cloud services
CO4	Orchestrate containerised workloads using Kubernetes
CO5	Provision and manage cloud infrastructure declaratively and automate delivery end to end

Assessment

This is a **micro-credential**, and it carries three components only.

COMPONENT	WEIGHT	INTERNAL OR EXTERNAL
Class assignment	50%	Internal
End-semester examination	30%	External
Class examination	20%	Internal

NOTE · THIS SUPERSEDES THE PATTERN PRINTED IN THE CURRICULUM DOCUMENT

The 2024 curriculum booklet lists a five-component scheme. That is no longer in force for micro-credential courses. The table above is what applies.

Class assignment, 50%

One deliverable, and the centre of gravity of the whole course: a small application, **containerised, deployed, declared as infrastructure code, and delivered by a pipeline**. It is not eight separate submissions. It is one system that you build up across the semester, and the practice labs are the steps.

STAGE	WHAT IT DEMONSTRATES	MARKS
Containerised application	Multi-stage build, pinned base, non-root, no baked secrets	10
Orchestrated	A Deployment and Service with probes, requests and limits	12
Declared as code	Infrastructure from a template, applied, converged, destroyed	12
Delivered by a pipeline	Commit runs tests, builds once, deploys the same artefact	12
Report and teardown proof	What you built, what it cost, and evidence it is all gone	4
Total		50

Submitted once, near the end of the semester, as a repository plus a report. The deadline and the exact submission channel are announced in class.

PITFALL · DO NOT LEAVE IT TO THE LAST FORTNIGHT

Each stage depends on the one before it, and each of the practice labs builds exactly one stage. Students who do the labs as they come have the assignment nearly finished by week 14. Students who skip them are attempting four unfamiliar technologies at once, under a deadline, with a live cloud account.

The practice labs

Eight labs, **not separately marked**, each building one stage of the assignment:

LAB	BUILDS	FEEDS
1	First instance, budget alert, clean teardown	Teardown proof
2	Object storage from the CLI and from code	Application
3	Containerise an application	Stage 1
4	A multi-container stack with persistent state	Stage 1
5	Kubernetes deployment, service, probes	Stage 2
6	Autoscaling under load	Stage 2
7	Infrastructure as code: provision, converge, destroy	Stage 3
8	A pipeline from commit to running deployment	Stage 4

They carry no marks of their own. They are how the 50 percent gets earned.

Class examination, 20%

One written examination during the semester, covering the syllabus to that point: cloud models and economics, fog and edge, virtualization, containers, and Kubernetes.

End-semester examination, 30%

Covers the whole syllabus including infrastructure as code, CI/CD and the two case studies.

Free-tier accounts

You will need one cloud account. All three major providers offer a free tier that is sufficient for every lab in this course.

PITFALL · SET A BUDGET ALERT ON DAY ONE

Before you create a single resource, set a spending alert at a small amount. The most common way a student incurs a real bill is leaving a load balancer or a managed database running over a holiday. Every lab ends with a teardown step for exactly this reason, and teardown is part of the marks.

Week-by-week plan

Weeks are numbered from the start of instruction, so this plan is reusable in any offering.

WEEK	LECTURES	LAB
1	What cloud is; the five characteristics; use cases	Account setup, budget alerts, first VM, then destroy it
2	Service models; deployment models; the economics	Compare on-demand and reserved pricing for a real workload
3	Storage: object, block, file. Networking and VPCs	Object storage from the CLI and from code
4	Security, identity, the shared responsibility model	Least-privilege roles; find and fix a public bucket
5	Fog and edge computing; latency budgets	Measure real latency to several regions
6	Hypervisor virtualization; type 1 and type 2	Build a VM image, measure boot time and overhead
7	Container virtualization: namespaces and cgroups	Containerise an application; layer caching
8	Images, registries, multi-stage builds, volumes	Multi-stage build; shrink an image; persist data. Class exam
9	Web application development on public cloud services	Deploy a web app to a managed platform service
10	Kubernetes: the reconciliation model; core objects	First deployment, service, and rollout
11	Services, ingress, probes, requests and limits	Expose it, add probes, break it deliberately
12	Scaling, rollouts, and when Kubernetes is the wrong answer	Autoscaling under load
13	Infrastructure as code: declarative, idempotent, state	Write, plan and apply a template; then destroy
14	CI/CD: pipelines, gates, artefacts, secrets	A pipeline that tests, builds and deploys on merge
15	Case studies: Echno, and a digital twin on Azure	Class assignment consultations

About the labs

The labs are practice, not separate submissions. Each one produces a piece of the class assignment, so the work is not thrown away.

NOTE · TEARDOWN STILL MATTERS, AND IS MARKED IN THE ASSIGNMENT

A lab that leaves resources running is incomplete. Provisioning something is half the skill; being able to remove it completely and prove that it is gone is the other half, and it is the half that keeps you employable. The assignment carries four marks for teardown evidence, and every lab ends with the step.

The two case studies

These are not textbook examples. They are systems I work on, presented with the architecture and the failures, and scrubbed of anything confidential: no credentials, no customer data, no commercial terms.

| Echno, a construction-intelligence platform

Infrastructure declared in OpenTofu, hosts configured with Ansible, applications in containers, a Kubernetes track with GitOps, and one set of roles serving three very different targets including a machine reachable only through a tunnel. The theme is the recurring engineering decision: **duplicate, or parameterise.**

| A digital twin on Azure

Everything declared in Bicep, custom domains with managed certificates, observability configured in a single call, and a pipeline that builds infrastructure for real on merge, asserts that it converged, and tears it down in the same run. The theme is what rigour costs and what it catches: a template that would never have converged, a teardown ordering that stranded a certificate, and a data service that answered a question it could not actually answer.

Tools you will need

```
# Debian / Ubuntu
sudo apt install git curl docker.io
sudo usermod -aG docker "$USER"      # log out and back in
```

TOOL	PURPOSE
Docker	Every container lab
<code>kubectl</code>	Kubernetes labs
<code>kind</code> or <code>minikube</code>	A local cluster, so you do not pay for a managed one
OpenTofu or Terraform	Infrastructure as code
A cloud CLI	<code>aws</code> , <code>az</code> or <code>gcloud</code>
<code>git</code>	Everything

Policies

Late submissions. Marked out of 80 percent within 48 hours; not accepted after that except with documented cause.

Collaboration. Discussing approaches is encouraged. Submitting work you did not produce is not.

AI tools. Permitted for learning and exploration. You must be able to explain everything you submit, including every line of a template or manifest. Cloud configuration produced without understanding is how expensive and insecure systems get built, so this matters more here than in most courses.

Secrets. Never commit a credential, in this course or ever. A key pushed to a public repository is scanned and abused within minutes. A submission containing a live credential scores zero and you rotate the key immediately.

Reading

The slide decks and notes are self-contained. If you want more:

- Buyya, Vecchiola, Selvi, *Mastering Cloud Computing*. The syllabus reference.
- Kocher, *Microservices and Containers*.
- Burns, Beda, Hightower, *Kubernetes: Up and Running*. The best Kubernetes introduction.
- Brikman, *Terraform: Up and Running*. Applies equally to OpenTofu.
- Your provider's own documentation, which is genuinely the best source and is free.

Getting help

Bring what you expected, what happened, the exact error, and what you have already tried. For cloud problems also bring the region, the resource type, and whether it ever worked. Screenshots of a console are much less useful than the CLI output.